Project 9 | MCA Projects 2026 | Section A

HEXARCH Study Guide

Instruction and Data Memory Hierarchy, explained from zero.

> **One sentence version: separate paths for instructions and data let a processor do two memory jobs in the same clock tick. A shared single-port memory has to do them one after another.**

What is in this guide

1. The big idea in plain words
2. Words you need to know
3. The tiny processor and its three-instruction program
4. The two designs side by side
5. Cycle by cycle: why 4 versus 6
6. The files: what each one does
7. The one rule that creates the difference
8. Tests, results and how to run everything
9. Review: what was checked, what is still open
0. Questions you may be asked
1. Video scripts for all five members

Team: Akshat Chauhan, Ajeesh Amreet, Vaibhav, Hardik Maheshwari, Mitul Bhatia. Phase I deadline: 16 November 2026, 11:59 PM IST.

Every number in this guide comes from a real simulation run (Icarus Verilog 13.0).

1. The big idea in plain words

A processor keeps asking memory two different questions:

- **"What should I do next?"** The answer is an **instruction**.
- **"What value am I working on?"** The answer is **data**.

Think of a kitchen

A cook needs a **recipe** (instructions) and **ingredients** (data).

Harvard: two counters

One counter hands out recipe steps. A different counter hands out ingredients. The cook can **read the next recipe step while grabbing an ingredient**. Nobody waits.

Shared: one counter

One window serves both. If the cook is collecting an ingredient, the next recipe step **has to wait** until the window is free.

What this project does

We built both memory designs in **Verilog** (a language for describing digital hardware), gave them the **same tiny processor, same program, same data, same total memory**, simulated them, and counted the clock cycles each needs.

> **The result**
> Same answer in both (the value 7 gets copied to a new place). Harvard finishes in **4 cycles**. Shared memory needs **6 cycles**, because two instruction fetches had to wait for the single port.

> **An honest limit, say this out loud if asked**
> This proves a point about *port contention* in a tiny model. It does **not** prove "Harvard is always faster". A program with no data accesses takes 4 cycles in *both* designs (we tested it).

Where it sits in the real world

Many small microcontrollers (PIC, AVR) use separate instruction and data memories. Big CPUs usually present one shared memory but use separate instruction and data *caches*. That is why it is called a "memory hierarchy" project. The optional lower-level shared memory from the brief is **deliberately left out**, which the brief allows.

2. Words you need to know

Word	Meaning in this project
Instruction	A one-byte command such as LOAD, STORE, HALT.
Data	The value being worked on. Here: the number 7.
Memory	A table of bytes you read by address. Instruction memory: 16 bytes. Data memory: 16 bytes.
Port	One "door" into a memory: one address in, one value out per cycle. A single-port memory can serve only one access per cycle.
Clock / cycle	The heartbeat. Things change on the **rising edge**. In simulation one cycle is 10 ns (a test setting, not a measured hardware speed).
PC (program counter)	4-bit counter holding the address of the next instruction to fetch.
Fetch	Reading the next instruction from memory.
IR (instruction register)	Holds the instruction that is running *now* while the PC already points at the next one.
ACC (accumulator)	One 8-bit register that temporarily holds a value. LOAD fills it, STORE empties it into memory.
Valid bit	`execute_valid` : "the IR holds a fresh instruction". Stops an old instruction from running twice when a fetch was blocked.
Harvard	Separate instruction and data memories with independent ports.
Shared / single-port	One memory, one port, instructions and data take turns.
Address decoder	In shared mode: picks the physical address. Instructions to 0-15, data to 16-31.
Stall / bubble	A cycle where the fetch is blocked, so nothing new enters the processor.
HALT	Instruction that stops the processor; it then makes no memory requests.
Testbench	Verilog code that runs the design and checks the answers automatically (`tb/tb_memory_compare.v`).
VCD / waveform	A recording of every signal over time that a viewer can plot.

3. The tiny processor and its program

Every instruction is one byte: two halves (nibbles)



Hex bytes in `programs/demo.hex` : **10 21 F0**

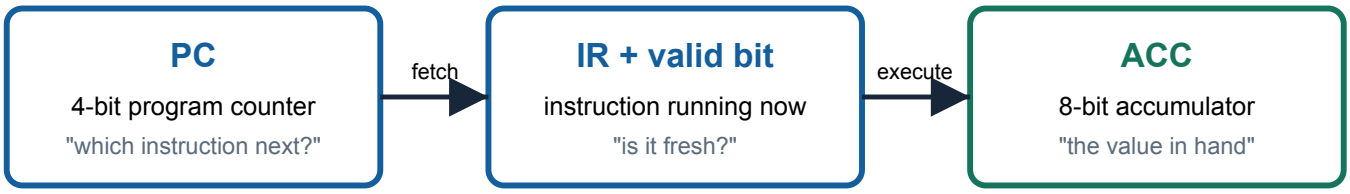
High nibble	Instruction	What it does
`1`	**LOAD a**	ACC ← data[a]
`2`	**STORE a**	data[a] ← ACC
`F`	**HALT**	Stop; no more memory requests
`0`	**NOP**	Do nothing
anything else	**illegal**	Stop and raise the `fault` flag

The demonstration program

```
LOAD 0     ; ACC = data[0]   (data[0] starts as 7)
STORE 1    ; data[1] = ACC   (now data[1] is 7 too)
HALT       ; stop
```

> It simply **copies the number 7 from data address 0 to data address 1**. Both designs must end with data[1] = 7 and ACC = 7.

The processor's three parts



"Fetch" really goes through memory: the PC supplies an address, the instruction memory returns the byte, and the byte is stored in the IR.

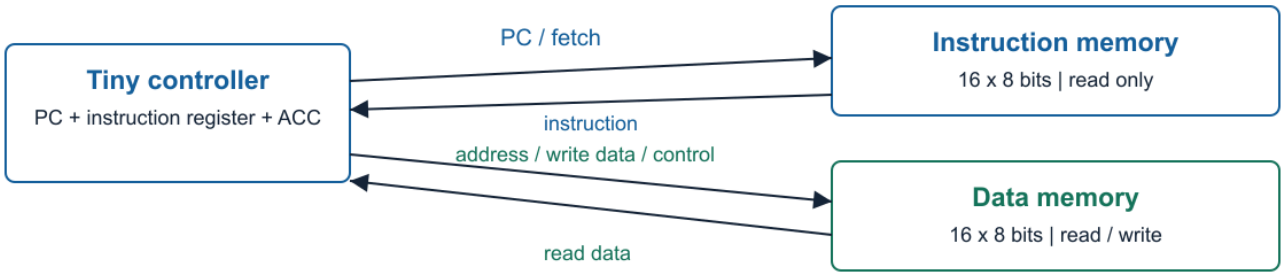
Having **one instruction register** is what lets the processor **execute the current instruction while fetching the next**. That overlap is the only pipelining in the design, and it is exactly what the Harvard memory makes possible.

4. The two designs side by side

One controller. Two memory organizations.

Both store 16 instruction bytes and 16 data bytes.

HARVARD: independent instruction and data ports



Fetch the next instruction while the current LOAD / STORE accesses data.

SHARED: one port must take turns



Addresses 0-15: instructions. Addresses 16-31: data. LOAD / STORE pauses fetch.

	Harvard	Shared single-port
Memory	16-byte instruction + 16-byte data	One 32-byte memory
Ports	Two independent	One, taken in turns
Addresses	Each memory starts at 0	0-15 instructions, 16-31 data (address decoder)
Fetch during LOAD/STORE?	Yes, same cycle	No: data wins, fetch is blocked, PC holds
Same for both	Controller, program, starting data, total capacity (32 bytes), clock period	

In the code, one parameter chooses the design: `SHARED_BUS = 0` is Harvard, `SHARED_BUS = 1` is shared (in `rtl/memory_system.v`). The same controller runs on both, so the only thing that differs is how the memory is wired.

5. Cycle by cycle: why 4 versus 6

Read each row left to right. Light box = **fetching** that instruction. Dark box = **executing** it.

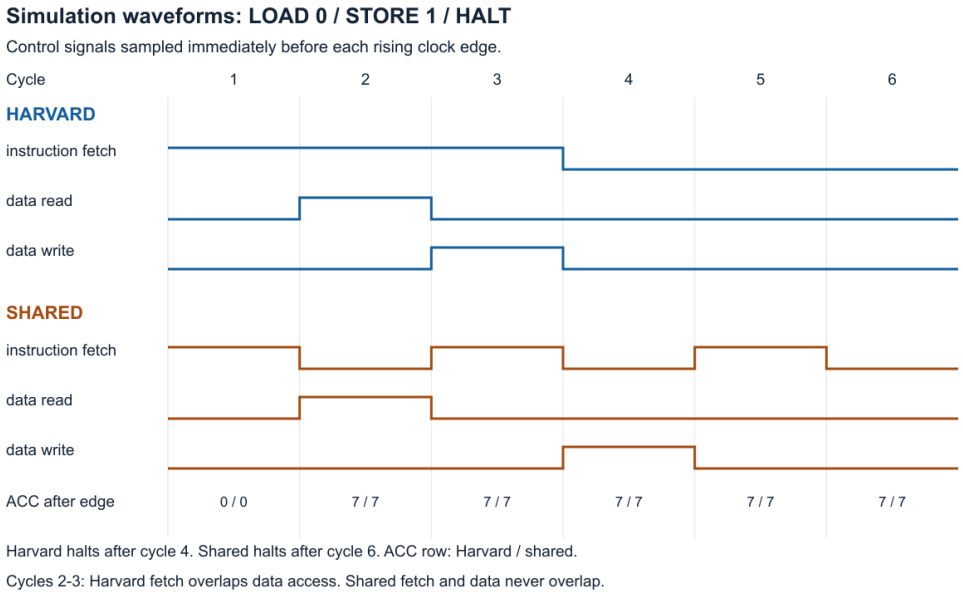
Harvard: 4 cycles

	1	2	3	4	5	6
Fetching	LOAD 0	STORE 1	HALT	none	stopped	stopped
Executing	empty	LOAD 0 reads 7	STORE 1 writes 7	HALT	stopped	stopped

Shared: 6 cycles

	1	2	3	4	5	6
Fetching	LOAD 0	BLOCKED	STORE 1	BLOCKED	HALT	none
Executing	empty	LOAD 0 reads 7	empty	STORE 1 writes 7	empty	HALT

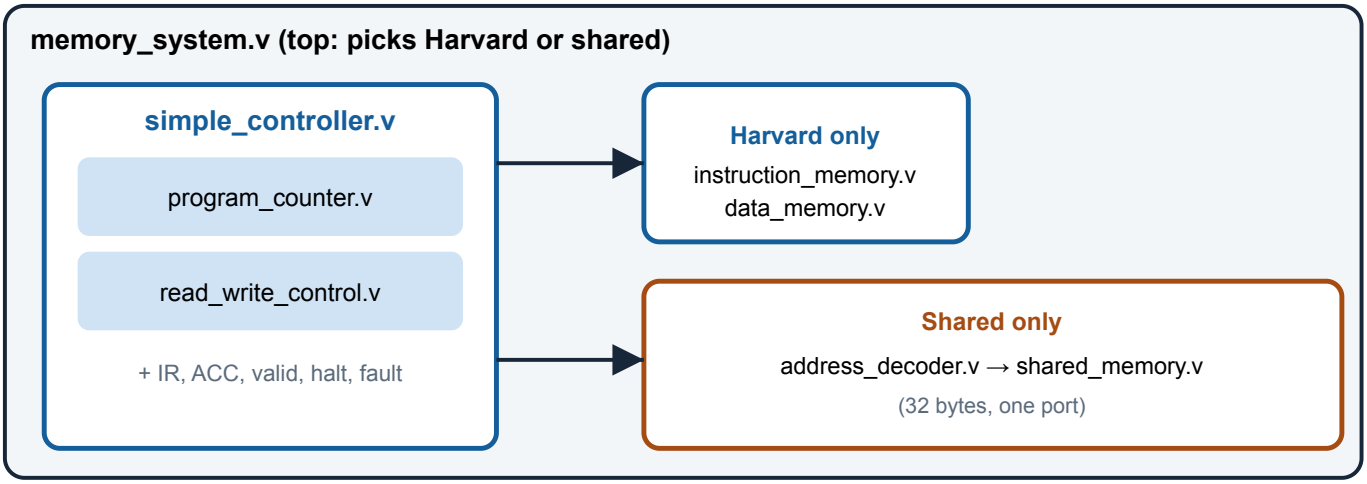
> **Cycle 2 is the key moment.** Harvard's fetch and data read signals are both 1 at once. In shared memory the fetch signal is 0, because the only port is busy reading data. The same thing happens in cycle 4 when STORE writes. Two blocked fetches = two extra cycles = 6 instead of 4.



Same story as lines going up and down. Blue (Harvard): fetch stays high while read and write pulse. Orange (shared): fetch drops every time read or write is high.

Why "empty" in cycles 3 and 5 (shared)? A blocked fetch leaves no fresh instruction, so the valid bit goes to 0 and the old LOAD is not executed again.

6. The files: what each one does



File (rtl/)	Job	Used in
`program_counter.v`	Counts up the instruction address, but **only when a fetch is accepted**.	Both
`instruction_memory.v`	16 bytes, loaded from `programs/demo.hex`. Read-only.	Harvard
`data_memory.v`	16 bytes, loaded from `programs/data.hex`. Read; write on the clock edge. Reset does not erase it.	Harvard
`read_write_control.v`	Decodes the instruction's top nibble into LOAD / STORE / HALT / illegal.	Both
`simple_controller.v`	The brain: holds IR, ACC, valid, halted, fault; decides when to fetch and when to read/write.	Both
`address_decoder.v`	One-line mux: data access gives address 16-31, otherwise 0-15.	Shared
`shared_memory.v`	32 bytes behind a single port; same read/write behaviour.	Shared
`memory_system.v`	Plugs the same controller into either memory arrangement.	Both

Other things in the repository

`tb/tb_memory_compare.v`	Runs both designs side by side, counts cycles, and **fails loudly** if anything is wrong.
`programs/*.hex`	The program (`10 21 F0 ...`) and the starting data (`07 00 00 ...`).
`results/`	Saved outputs: cycle table, CSVs, waveforms (`.vcd`).
`docs/` , `output/pdf/`	Explanations, requirement checklist, IEEE report (LaTeX source + PDF), figures, video script.
`Makefile`	Shortcuts: `make demo` , `make test` , `make figures` , `make report` .

7. The one rule that creates the difference

Open `rtl/simple_controller.v`. Everything above comes from four lines:

```
assign data_read_enable  = running && execute_valid && load;
assign data_write_enable = running && execute_valid && store;
assign fetch_enable = running && !stop_now && !(SHARED_BUS && data_access);
```

- **LOAD / STORE** raise a data read / write signal while their instruction is running.
- **Fetch** is allowed unless the processor is halting, **or** (shared mode *and* a data access is happening).
- In Harvard mode `SHARED_BUS` is 0, so that last part is always false: fetch never waits.

What blocking a fetch does

1. `fetch_enable` = 0, so the PC does not advance (it only counts when a fetch is accepted).
2. The valid bit is loaded with 0, so next cycle nothing runs twice.
3. Next cycle the data access is over, `fetch_enable` = 1, and the delayed instruction is fetched.

Timing rules in one box

> 1. **Reads** are instant (combinational). **LOAD** captures the value at the rising edge. **STORE** writes at the rising edge.
> 2. A shared data access blocks the fetch, so the PC holds for one cycle.
> 3. **Reset** clears the controller and suppresses all requests, but keeps memory contents. **HALT** stops all further requests.

The address decoder (shared mode)

```
physical_address = data_select ? {1'b1, data_address}      // 16..31 : data
                               : {1'b0, instruction_address}; //  0..15 : instructions
```

The top bit of the 5-bit address says "instruction side" (0) or "data side" (1). That is the whole mapping.

Known limits of the model (good to know before an interview)

- Programs must fit in 16 bytes and finish with HALT. The 4-bit PC wraps around otherwise.
- No jumps or branches, no arithmetic. Only LOAD, STORE, HALT, NOP.
- Memory reads are combinational and writes happen on the rising edge: this is a simplified model, not a specific chip.
- The 10 ns clock is a simulation setting. FPGA resources and real frequency were **not** measured.

8. Tests, results and how to run everything

Run it yourself (needs Icarus Verilog, Make, Python 3)

```
cd MCA_Project
make demo     # the 6-cycle table, prints DEMO PASSED
make test     # all six scenarios, prints ALL TESTS PASSED (6 scenarios)
```

Install on macOS: `brew install icarus-verilog`. On Ubuntu/Debian: `sudo apt-get install iverilog make python3`.

The six scenarios and the results we measured

#	Scenario	Why it exists	Harvard	Shared
1	**demo**: LOAD 0, STORE 1, HALT	The main comparison	4	6
2	**address boundaries**	Uses data addresses 0 and 15; a STORE right before a LOAD of the same address	6	10
3	**no data access**: two NOPs, HALT	Shows the gain depends on the program	4	4
4	**HALT only**	Stops immediately, no traffic	2	2
5	**illegal opcode**	Stops with the fault flag set	2	2
6	**reset and restart**	Reset mid-program, memory kept, restart, longer copy chain	8	14

Cycle count starts at the first clock edge after reset is released and includes the HALT cycle.

What the testbench checks automatically

✓ Every data word ends with the expected value, in both designs.
✓ Instructions are never changed.
✓ Each LOAD / STORE happens exactly once (no repeats).
✓ PC advances only on a fetch, and holds during a blocked fetch.
✓ In shared mode, fetch and data access never happen in the same cycle.
✓ After HALT nothing touches memory; ACC is held.

Where the evidence lives

`results/demo_output.txt`	The readable cycle table from `make demo` (F/R/W = fetch / read / write).
`results/test_output.txt`, `summary.csv`	All six test results.
`results/demo.vcd`	Waveform. Open in a viewer such as GTKWave (not installed on this Mac). Look under `tb_memory_compare.h` (Harvard) and `.s` (shared).
`output/pdf/HEXARCH_Phase_I_Report_Draft.pdf`	2-page IEEE-format report draft.

9. Review: what was checked, what is still open

Checked and correct

✓ Clean rebuild: **6 of 6 scenarios pass**, and the demo prints DEMO PASSED.

✓ Compiler (`iverilog -Wall` , v13.0): **zero warnings**.

✓ Cycle table in the guide, README and report matches the real simulation trace (4 vs 6).

✓ All design claims were traced through the code (PC hold, valid bit, shared priority, reset keeps memory, HALT stops traffic).

✓ Report PDF is rebuilt from its LaTeX source and the text is identical. Two pages, no layout errors.

✓ GitHub repo exists (private) and its latest commit equals the local one.

✓ No junk files tracked (`build/` , `tmp/` , `.DS_Store` are ignored).

Fixed during this review

- `docs/VIDEO_SCRIPT.md` said the GitHub repo was "not yet published". It is. Line corrected.
- In the architecture diagram, the "read data" label touched the Address decoder box. Moved and regenerated.
- Added word-for-word scripts: `docs/MEMBER_SCRIPTS.md` .

Still open (not code bugs, these decide your marks)

! **Video not recorded** (15 marks). Max 5 minutes, all five names, working demo.

! **Git history is one commit, by Mitul.** The brief rewards meaningful *individual* history. Each member must make a real change from their own account.

! Report section "Individual Contributions" is still **proposed**. Replace with what each person really did, then delete the draft notice and rebuild.

! Video link not yet in `VIDEO.md` .

! Evaluators and teammates need **access to the private repository**.

! Submission form not submitted. Deadline **16 Nov 2026, 11:59 PM IST**.

! Two Microchip citations in the report could not be re-opened from this machine (the pages timed out). Open both links once and confirm they say what the report claims. The third (Icarus Verilog) was confirmed.

! README and docs say starter code was made with AI assistance. Keep that honest and have each member be able to explain their own part.

> **Verdict.** The technical project is correct and complete for its stated scope. What remains is people work: real individual commits, the video, and submission.

Suggested real contribution for each member (small, honest, doable)

Member	Own this part	A real improvement they could make
Akshat	Instruction memory, PC	Add a test: a program longer than 3 instructions, or one that checks PC wrap-around behaviour
Ajeesh	Data memory, read/write control	Add a test that reset keeps memory; document the read/write timing in a short note
Vaibhav	Shared memory, address decoder	Add an annotated example showing the physical address for LOAD 0 and STORE 1 in shared mode
Hardik	Testbench, waveforms	Add a new scenario with its measured cycle count and a short waveform note
Mitul	Integration, report, video	Final report update, video link in `VIDEO.md` , repository access check

Never split existing files into fake commits or commit as someone else.

10. Questions you may be asked

Question	Short answer
What is a Harvard architecture?	Separate memories and buses for instructions and data, so both can be accessed in the same cycle.
What is the shared alternative called?	Von Neumann style: one memory for both. With a single port, only one access per cycle.
Why 4 cycles versus 6?	Two of the program's instructions (LOAD, STORE) use data memory. In shared mode each blocks the next fetch for one cycle: 4 + 2 = 6.
Is Harvard always faster?	No. With no data accesses both take 4 cycles (our no-data test). It helps when fetch and data access would compete.
Why is there a valid bit?	When a fetch is blocked the IR still holds the old instruction. The valid bit says "nothing new", so it does not run twice.
Why only one instruction register?	That is enough to overlap "execute current" with "fetch next", which is all the demo needs.
What does reset do?	Clears PC, ACC, IR, valid, halted, fault and stops memory requests. Memory contents are kept (tested).
What happens with an unknown opcode?	The controller halts and sets the `fault` flag (tested).
What if a program has no HALT?	The 4-bit PC wraps after 16 instructions and the program repeats. The unused image bytes are filled with HALT (`F0`) to avoid this in our programs.
How do you know both designs are fair?	Same controller, same program and data, same 32-byte total, same clock period. Only the memory wiring differs.
How did you verify correctness?	An automatic testbench with six scenarios checks every data word, access counts, PC behaviour, port ownership and halt behaviour.
What is not covered?	Caches, branches, a full instruction set, synthesis or FPGA measurement, real clock speed. The optional lower-level shared memory is omitted.
Did you use AI?	The repository states starter code and drafts were prepared with AI assistance. Be ready to explain your own part and your own commits honestly.
What does F/R/W mean in the output?	Instruction fetch / data read / data write.

If they ask you to open a file and explain

- **Waveform:** open `results/demo.vcd`. Show `clk`, `pc`, `instruction_register`, `execute_valid`, `fetch_enable`, `data_read_enable`, `data_write_enable`, `accumulator`, `halted` for both `h` and `s`.
- **Code:** go straight to the `fetch_enable` line in `simple_controller.v`, then `address_decoder.v`.

11. Video scripts for all five members

Total about 470 words, roughly **3 min 35 s** at a relaxed pace (the limit is 5 minutes). Read your part twice, then practise it with the screen action. Keep all five names visible on screen. The same text is saved in `docs/MEMBER_SCRIPTS.md`.

0:00 - 0:20 | ALL FIVE | OPENING

Show: title slide or README with all five names.

Akshat: "Hi, I'm Akshat Chauhan."
Ajeesh: "I'm Ajeesh Amreet."
Vaibhav: "I'm Vaibhav."
Hardik: "I'm Hardik Maheshwari."
Mitul: "And I'm Mitul Bhatia. We are Team HEXARCH, Section A. Our project is Instruction and Data Memory Hierarchy."

0:20 - 0:55 | AKSHAT | THE IDEA AND THE INSTRUCTION PATH

Show: `docs/figures/architecture.png`, pointing at the Harvard half.

"Instructions say what to do. Data is the value being worked on. We compare two ways of storing them. In a Harvard design, instructions and data have separate memories and ports, so the processor can fetch the next instruction while the current one uses data. In a shared design, both sit in one memory with one port, so they take turns. The program counter, or PC, picks the next instruction, and one instruction register holds the instruction that is running now."

0:55 - 1:30 | AJEESH | THE PROGRAM AND DATA MEMORY

Show: the three-line program in the README, then `programs/demo.hex`.

"Our program is LOAD 0, STORE 1, and HALT. In memory these are the bytes 10, 21 and F0: the first digit is the operation, the second is the data address. Data address 0 starts with the value 7. LOAD copies the 7 into the accumulator, a single eight-bit register. STORE writes it into data address 1. HALT stops. Reads are immediate; the writes happen on the rising clock edge. Both designs must end with 7 at data address 1."

1:30 - 2:00 | VAIBHAV | WHY SHARED MEMORY HAS TO WAIT

Show: the Shared half of the diagram, then `rtl/address_decoder.v` and the `fetch_enable` line in `rtl/simple_controller.v`.

"Both designs hold 32 bytes in total. In the shared one, the address decoder maps instructions to addresses 0 to 15 and data to 16 to 31, behind one port. When LOAD or STORE needs that port, data wins: the next fetch is blocked, the program counter holds, and the instruction is fetched a cycle later. In the code this is one rule: fetch is switched off whenever shared mode has a data access."

11. Video scripts (continued)

2:00 - 3:00 | HARDIK | LIVE DEMO

Run: `make demo`. Then show `docs/figures/waveforms.png` or open `results/demo.vcd`.

"Let's run make demo. Each row is one clock cycle; F, R and W mean fetch, data read and data write. In cycle 2, Harvard fetches STORE while LOAD reads data, so fetch and read are both 1. Shared memory cannot: its fetch is 0, because the single port is busy. In cycle 3, Harvard fetches HALT while STORE writes the 7. Harvard executes HALT in cycle 4, so it finishes in four cycles. Shared memory fetches STORE in cycle 3, writes in cycle 4, fetches HALT in cycle 5 and executes it in cycle 6. Six cycles; the extra two are the two blocked fetches. Both designs end with 7 at data address 1."

3:00 - 3:30 | MITUL | VERIFICATION AND WRAP-UP

Run: `make test`. Show the six PASS lines, the report PDF and the repository.

"Finally, make test runs six scenarios, including boundary addresses, HALT only, an invalid opcode and a mid-program reset. All six pass. The program with no data accesses takes four cycles in both designs, so separate paths only help when a fetch and a data access would otherwise compete. This is a simulation of one small model, not a universal speedup claim. Our repository has the Verilog, testbench, diagrams, waveforms, results and an IEEE-format report. **[Say the team's real remaining work here.]** Thank you."

Recording checklist

- ☐ Everyone has run `make demo` and `make test` on their own laptop first.
- ☐ Terminal font large enough to read; run `clear` before `make demo`.
- ☐ Open the diagram, waveform image and the three code files in advance, in tabs.
- ☐ Each person says their own full name clearly in the opening.
- ☐ Total length at or below 5:00. Upload, then test the link from a different account.
- ☐ Put the final link and duration into `VIDEO.md`.

> **Replace the bracket in Mitul's last paragraph** with the true state on recording day, for example which contributions each person has finished. Do not claim work that is not in the repository.